

SCUDEM Problem B: An AI Ouroboros

Pranav Kulkarni and Sharad Patel

MAP 4484: Modeling in Mathematical Modeling

Dr. Tracy Stepien



1. Background

In recent years, artificial intelligence (AI) systems have grown in number, size, ability, and popularity. Specifically, large generative models such as large language models (LLMs) have become highly integrated in the digital ecosystem. Popular models such as ChatGPT have become adept at producing text, images, code, and other creative content at unprecedented scales. Over time, more AI-generated output enters the digital environment and inevitably shapes the data environment from which future models will learn. In other words, a larger portion of the data new generations of AI models are trained on becomes output from past AI models. This positive feedback loop has been theorized to lead to a phenomenon called AI model collapse. In this degenerative process, the recursive training on AI-generated outputs degrades new models' ability to accurately represent the original underlying data distribution.

This issue of model collapse when models are trained on data produced by previous generations of models was introduced and analyzed by Shumailov et al. (2024). In early model collapse, these models first lose the ability to capture lower-probability events (the “tails” of the original data distribution). This occurs as subsequent models are not able to capture the true data distribution effectively, causing errors to compound over generations of recursive training. Over time, important statistical features of the model are lost, and in later stages, the model collapses to a Dirac delta function, leading to severe degradation of model performance. This concept of recursive training and loss of model characteristics is visualized in Figure 1 below.

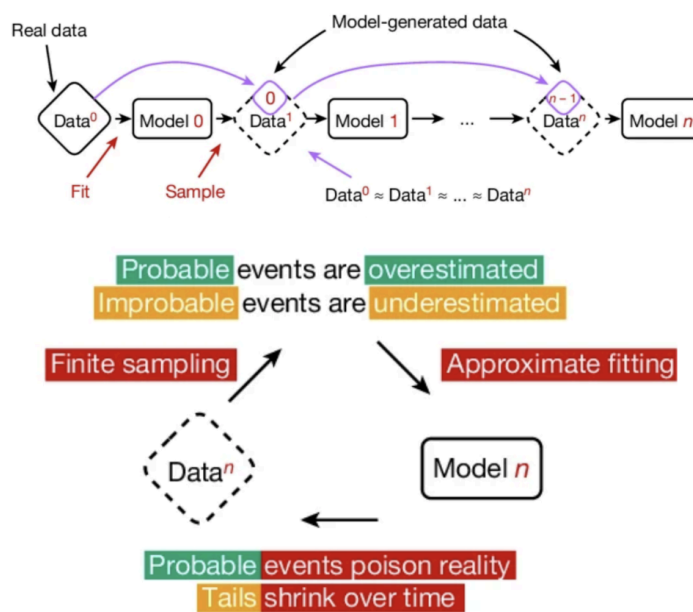


Figure 1 - adapted from Shumailov et al., (2014)

Mathematically, this collapse is related to three sources of error: statistical approximation error, functional expressivity error, and functional optimization error. Statistical approximation error is based on the idea that information regarding rare events is lost as the size of the sample data set is finite. Functional expressivity error is based on the idea that neural networks and models in general cannot perfectly capture the complexity of all distributions, especially in high-dimensional spaces. Functional optimization error emerges due to limitations in the

algorithms used to train models, such as stochastic gradient descent. Even if these errors are individually small, recursive training on model-generated data amplifies them, causing the learned distribution to shift away from the true one.

The paper provided by SCUDEM has empirical results that support these ideas. Using language models fine-tuned on the WikiText-2 corpus, Shumailov et al. showed that successive generations produce increasingly homogenized outputs that have shrinking variance with growing repetition. At the same time, these models start generating a higher frequency of nonsensical output that diverges from natural human language patterns. These experiments demonstrate both early-stage collapse, in which low-probability events vanish, and late-stage collapse, in which the model converges toward a near-degenerate distribution.

“SCUDEM Problem B: An AI Ouroboros” recognizes the risk of model collapse, especially given the recent flood of AI-generated content into the Internet; a 2024 Europol report predicted that ~90% of publicly available online content could be AI-generated within the next year.

Problem B also recognizes that in the real-world, many different AI systems coexist with varying architectures, training data, and outputs. In reality, the newer models are trained on data that contains a mixture of human content, AI-generated material, and human-altered AI-generated content. The central question posed by the SCUDEM prompt asks how multiple different AI models interact and are affected in the presence of this aforementioned sample dataset. We are asked to analyze long-term patterns, how model quality and the rate of deterioration are affected, and whether the presence of multiple models producing different outputs mitigates total model collapse.

Together, the literature and the SCUDEM problem description highlight an urgent need to model collapse in a multi-model environment. This will help answer questions about the stability of machine learning systems trained on public data and the long-term sustainability of generative AI technologies. Our model aims to effectively capture these interactions to clarify which conditions lead to stability and collapse and what interventions could be useful in preserving model quality over time.

2. Model Development

To understand recursive model collapse, we must capture 2 distinct type of failure modes: early stage collapse, which is the erosion of rare events and the loss of distribution richness, expressed with the loss of tails in a distribution, and late-stage collapse, which is where the model distribution eventually degenerates into a delta Dirac function (sharp peak), as seen by Shumailov et al. (2024). They showed that in a single model, training of synthetic data eliminated the statistical tails initially before collapsing to a narrow representation of the initial distribution.

Our project adapted the theoretical framework set up Shumailov et al. (2024) to including the degradation when including several unique multiple AI models by creating two complementary mathematical models:

1. A modified Markov-chain ecosystem model (Model 1) designed to capture early-stage collapse through information-theoretic degradation under recursive mixed human–AI training.
2. A layered discrete distribution model (Model 2) designed to capture late-stage collapse and structural degeneration in a task-complexity space.

Together, these models form a framework to understand the short and long term results of recursively training in a situation where multiple AI models interact with each other and with human data.

2.1 Shared Assumptions and Modeling Philosophy

Both models were constructed with shared assumptions, the first being collapse arises purely from statistical approximation error. Shumailov et al. (2024) mentioned the 3 ways a model can collapse: statistical approximation error, functional expressivity error, and functional approximation error. For the sake of simplicity and following the methods described by Shumailov et al., we omitted a study of model collapse due to functional expressivity and functional approximation errors. Essentially, we are focusing on the concept that recursive sampling compounds create statistical bias as information regarding rarer events are lost. We therefore model the collapse mechanism in isolation from any architectural limitations of the model itself.

The second major assumption is that human data P_0 is representative of an anchor of fidelity when it comes to true data. We are able to tweak parameter α to increase number of synthetic samples in Model 1.

The third principle is that multiple AIs interact via shared training data. The problem we wanted to tackle was regarding how an ecosystem of AIs progresses. Due to this, each model has M distinct learners, each of which contributes synthetic data to future generations.

The fourth and final principle we are working with is that recursive feedback is essential. To model collapse by training AI on synthetic data, the data that is generated by the model is used to train the next generation of models.

2.2 Model 1: Modified Markov-Chain Ecosystem Model

The motivation and goal of this model is to capture and understand early phase collapse, where the distribution resembles the real human data P_0 . However, the rare, low probability events (the tails of the distribution) are disappearing, as a byproduct of finite sampling statistical bias. As shown in Shumailov et al. (2024), this is the first measurable sign of model collapse.

To construct this model, we represent each AI model i 's output at generation n , $Q_{i,n}$ as a probability vector of V categories. At any generation n , the aggregate AI output is can be modeled as such:

$$Q_{AI,n} = \frac{1}{M} \sum_{i=1}^M Q_{i,n}$$

This is just that at any given generation, the overall output distribution is the average of each of the models' own unique distribution. Then, in order to create the distribution that the next generations samples from, we combine this synthetic output with the human data:

$$P_n = (1 - \alpha)P_0 + \alpha Q_{AI,n}$$

Finally we have the output of the $n + 1$ generation $Q_{i,n+1}$ by having each model train on the dataset P_n to get an equation $Q_{i,n+1} = F_i(P_n)$, where F_i is the transformation via model i . This model reflects what was done in the 1 model setup shown by Shumailov et al. (2024), with the addition of averaging the output of each model to account for the interaction between several AI's. This can allow us to model the effect of increasing the number of AI to see how the model progresses.

Following this setup, we can turn this model into a Markov-type model, where the current output distribution for each model just depends on the outputs of the models in the previous generation.

We can plug in $Q_{AI,n}$ being the average of all the model outputs at generation n into the P_n equation to get:

$$P_n = (1 - \alpha)P_0 + \frac{\alpha}{M} \sum_i \mathcal{F}_i(P_{n-1})$$

Equating the right and side of the equation to be a Markov-type operator:

$$T(P) = (1 - \alpha)P_0 + \frac{\alpha}{M} \sum_{i=1}^M \mathcal{F}_i(P)$$

After these steps, we can get write the dynamics compactly as $P_n = T(P_{n-1})$, obtaining a simple yet complex recursive model to replicate the effects of training several AI's on synthetic data created by each other. This exists as a markov model as P_n is a function of and only depends on P_{n-1} .

2.3 Model 2: Layered Discrete Distribution Model

To effectively model late-stage collapse, we created a layered discrete distribution model. This model was designed with an emphasis on the interaction of multiple distinct models in the environment. To do this, we defined a discrete task-space defined by a binomial distribution to represent various “tasks” the models perform. This represents the fact that different AI models in the real-world have different roles; for instance, some models may focus on text generation while others focus on image generation. To do this, we define a set of n discrete tasks indexed by $T = 1, \dots, n$. Their initial frequencies follow a binomial distribution π centered around medium-index tasks:

$$\pi(T = t) = \frac{N!}{(N - t)!t!} p^t (1 - p)^{N-t} = \frac{N!}{(N - t)!t!} 0.5^N$$

This provides a realistic, unimodal baseline where central tasks are more common while “complex” tasks are infrequent, a concept which will be elaborated upon further in this section. A binomial distribution is plausible here due to functional expressivity error; although we are not directly focusing on the effects of this error on model collapse, this error suggests that no distribution is capable of accurately capturing the original data distribution. Thus, we selected the binomial distribution for convenience and its ability to model real world tasks.

Each task is associated with a complexity distribution over discrete levels $X = 1, \dots, 5$. Defining complexity in this way allows us to differentiate between AI models that perform simpler versus more complex tasks. Complexity is modeled as a geometric distribution P :

$$P(X = x|T = t) = (1 - p)^{x-1} p = (\pi(T = t))^{x-1} (1 - \pi(T = t))$$

Notably, the geometric distribution for each task depends on the π -space distribution for tasks. This implies that the higher the complexity of the task, the rarer the task is. This allows us to take into account common and relatively simple AI models as well as rarer and highly specialized AI models, allowing our model to be more representative of reality.

The stochastic process by which the π -distribution is transformed is given by the following transformation:

$$\pi_i(T = t) \rightarrow P_i(X, T = t) \rightarrow D_i(t) \rightarrow P_{i+1}(X, T = t) \rightarrow \pi_{i+1}(T = t)$$

Here, we start with a task distribution at generation i and use it to help determine each task's complexity parameters. Next, we generate a dataset D_i by using the geometric distribution P_i to sample M synthetic complexity values. This essentially introduces the statistical approximation error as described by Shumailov et al. D_i is then used to fit a new geometric distribution using Maximum Likelihood Estimation, which represents model learning. At this step, the proportions of sampling data coming from AI-generated or human-generated data can be adjusted. Finally, the task space π_{i+1} is regenerated with the updated geometric distributions. This process works to model late-stage model collapse as errors introduced in complexity distributions generated from learning with a dataset that is at least partly made of AI-generated data are fed back into task distributions.

This model gives us the flexibility of defining the number of models, their tasks, and their complexities, the number of generations the models undergo learning, and the specific makeup of the sampling dataset.

2.3.1 Additional Model 2 Assumptions

In addition to the aforementioned shared assumptions between both models, Model 2 is used with additional assumptions given its complexity. As discussed in section 2.3, we assumed that tasks are distributed within the task space according to the Binomial distribution and that each task is in the same distribution family (Geometric). We also assume that all AIs use the same size of training dataset and that they are all trained the same way. Additionally, the literature estimated that approximately 50% of online data currently is generated from humans (Thompson et al., 2025). We thus assumed an equal proportion of AI-generated and human-generated content in the sampling dataset.

3. Results

This section presents the output of both models after we simulated them using MATLAB. For Model 1, the findings came in the form of the behavior of KL divergence and tail coverage across generations under different ecosystem configurations. For Model 2, the output is instead based on how the task-space and task-complexity distributions evolve under recursive training.

3.1 Results for Model 1: Modified Markov-Chain Ecosystem Model

Model 1 simulates M AI models undergoing recursive training based on the equations established in section 2. The results are visualized by two metrics computed at each generation:

- Tail Coverage: Ratio of tail mass in P_n compared to P_0
- KL Divergence: $KL(P_0||P_n)$, characterized by the quantifying drift away from human data

The overall output is a time series trajectories for each of the 2 metrics over 40 generations, all averaged over 6 monte carlo trials.

3.1.1 Single vs. Multi-Model Behavior

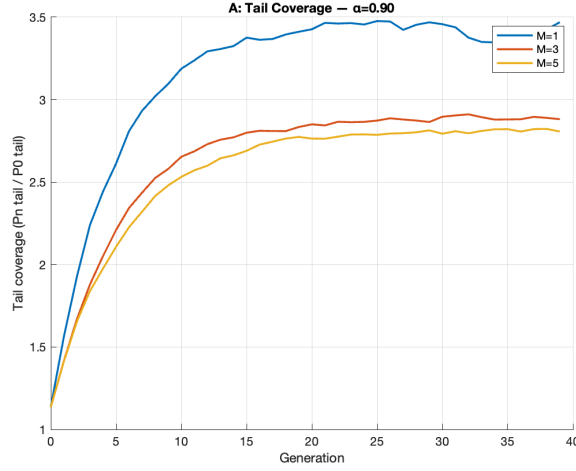


Figure 2 - Retention of tail information as model generations progress

This figure shows the evolution of trail coverage cross generation (up to 40) for different numbers of models. The curve is a result of the the codes calculation of:

$$\text{Tail} = \frac{\sum_{x \in \text{tail}} P_n(x)}{\sum_{x \in \text{tail}} P_0(x)}.$$

The trajectories plotted is the averaged output over all 6 trials for each M.

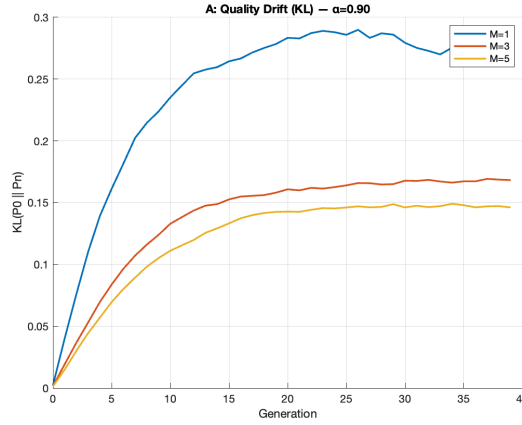


Figure 3 - Ability of model(s) to accurately describe initial data as generations progress for $\alpha = 0.90$

This figure shows the KL drift curve based on the equation $KL(P_0 \| P_n)$. This curve tells us how the average distribution drifts away from the original human distribution as the generations go on. The trajectories for this figure as well is averaged output over all 6 trials for each M.

3.1.2 Effect of the Synthetic-Data Fraction

Below, represented by figure 3 and 4, we examine the quality drift and tail coverage with a fixed number of AI models, however changing the amount of human data that helps build the next generations distribution. The same standard of the output being the averaged output over all 6 trials for each alpha value still holds.

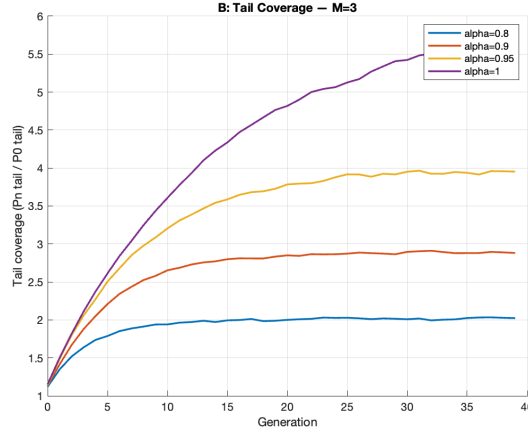


Figure 4 - Retention of tail information as model generations progress

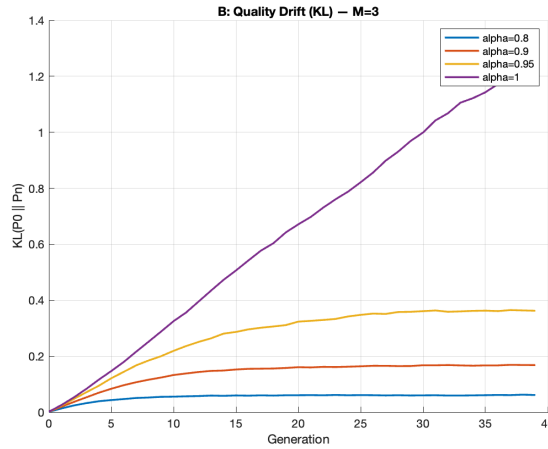


Figure 5 - Ability of model(s) to accurately describe initial data as generations progress for $M=3$

3.2 Results for Model 2: Layered Discrete Distribution Model

To verify that the base case of our model was accurate, we first simplified the situation to a single AI model. We defined a task space of size 40 with a single AI trained on Task 15 for 10 generations with a training data set size of 300. In line with our assumptions, we set the proportion of the data coming from AI-generated content as well as human-generated content to 0.5. It is important to note that the distribution of data coming from human-generated content matches the ideal task-space distribution defined initially. This is due to the complexity of the Tasks; as more complex Tasks are less rare in the binomial distribution, it follows that human-generated data follows this same trend; in other words, more “complex” and niche human-generated data is less represented in the distribution of sampling data. The results of the simulation with the above settings is shown in Figure 6 below:

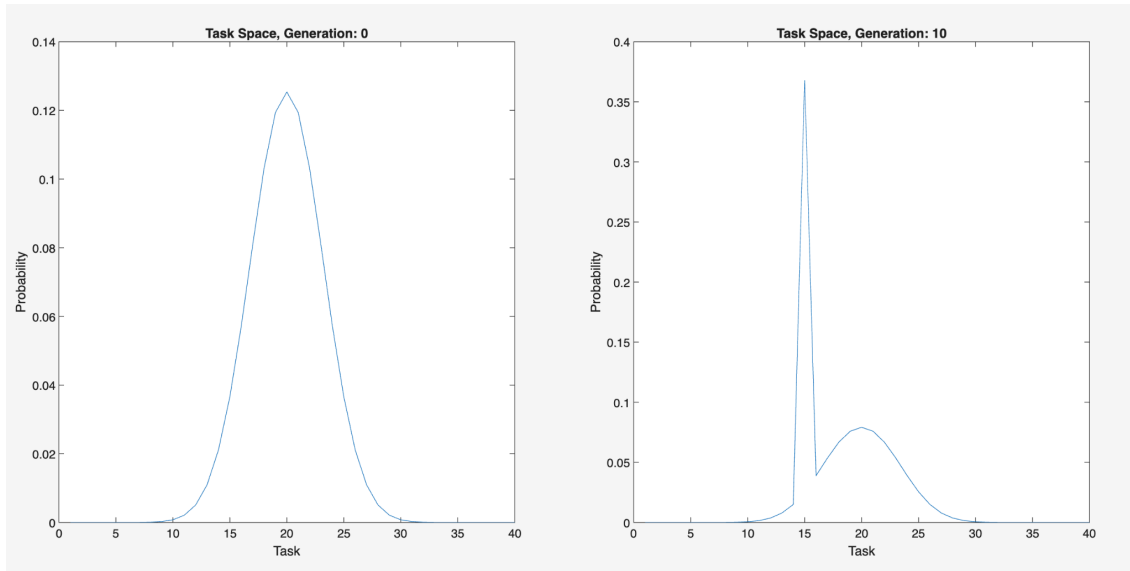


Figure 6 - Single AI model trained on Task 15 with 50% AI-generated input data

The graph at Generation 0 represents the initial distribution of the Task Space. Figure 6 shows that after just 10 generations, there is a “spike” formed around Task 15. In other words, a Dirac delta-type distribution forms localized around Task 15. This implies that the data distribution is collapsing around Task 15, which is consistent with the findings from Shumailov et al. for single model decay.

Next, we extended the simulation to include 30 AI models, with each model instructed to train on its own corresponding Task from Task 35 to Task 65. This was done in a task space of size 100 and the results of this simulation are shown in Figure 7:

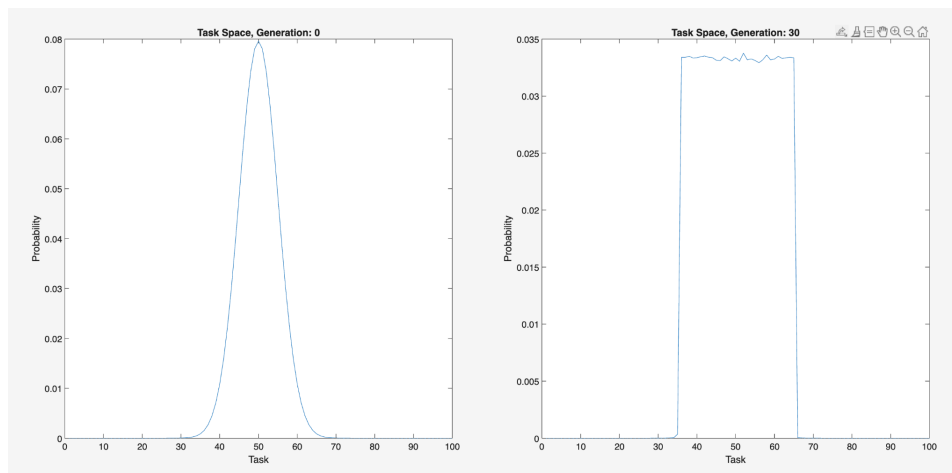


Figure 7 - AI models trained on Tasks 35-65 with 50% AI-generated input data

The resulting distribution at Generation 30 seems to be uniform. However, it is more likely that a delta Dirac-type “spike” is being created at each Task from 35-65, resulting in a final data distribution that is solely representative of what Task each AI model is instructed to train on. This idea can be verified by choosing non-consecutive tasks within the distribution. For instance,

Figure 8 shows a simulation ran for 100 Generations with 3 AI models trained on Tasks 20, 35, 41:

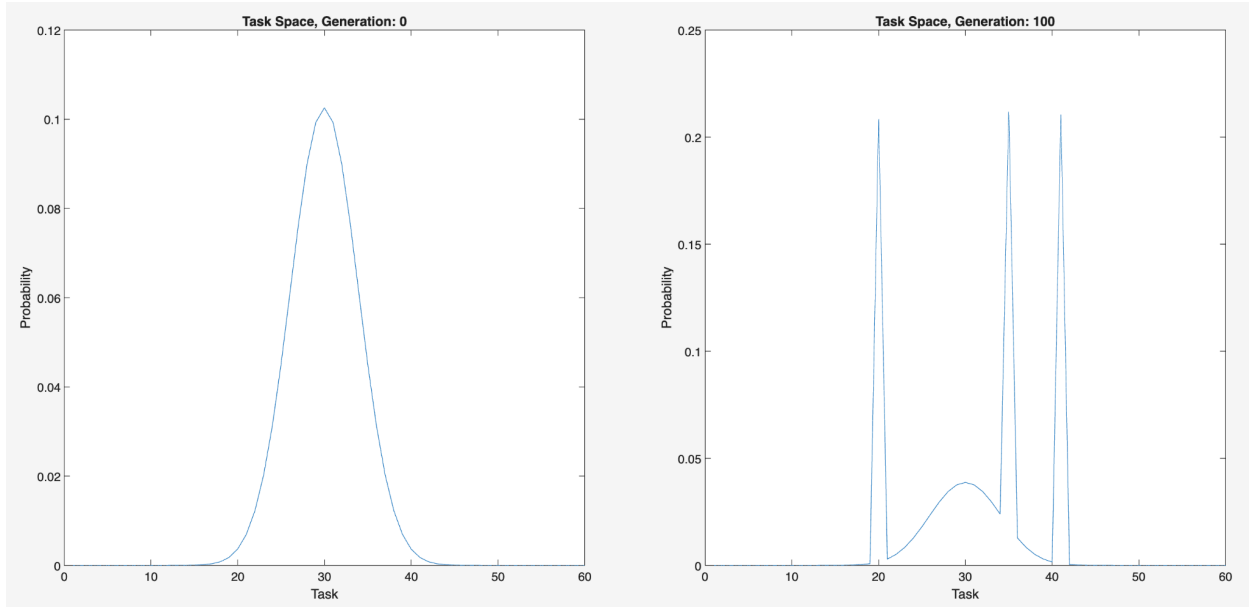


Figure 8 - AI models trained on Tasks 20, 35, 41 with 50% AI-generated input data

This supports the idea that the distribution collapses to a delta Dirac function at each individual Task. In other words, Model 2 suggests that in a digital environment where AI models are given specific tasks to train on, the resulting distribution of data within the environment collapses to a distribution characterized mainly by those Tasks. There are also remnants of the binomial distribution which come from the fact that human-generated data is still present in the sampling data. However, the probability of Tasks that are only part of this distribution significantly decreases over time, suggesting that only the Tasks that the AIs were defined to train on are relevant as generations occur.

4. Conclusions

In this work, we develop and analyze two complementary models to represent the same phenomena: as an AI model learned based on synthetic data or data it has produced before, we are able to capture and see 2 types of decay, an early-stage information-theoretic collapse and a late-stage structural collapse. We wanted to expand this framework to see if using multiple AI models trains on both synthetic and human data delays or prevents any information loss and decay, with the goal of mimicking modern AI ecosystems.

With Model 1, the Modified Markov-Chain Ecosystem Model, we saw how a Markov chain like model was able to induce systematic drift away from the true and human distribution P_0 . We were able to notice the signs of early stage model collapse in the output of this model, represented by seeing how the the trails degrade due to sampling noise, fidelity variation, and compounding recursive updates. However, even though every simulation had this tail degradation and quality drift from the original distribution, we did see that using more of the human distribution for each iteration reduced the degradation of the model. More importantly, we see that increasing the number of independently-trained models M slows the degradation down as well, with a massive reduction in tail loss and quality drift. This is through the fact that

each of these models have a unique distribution and samples based on its own separate transformation. Having this diversity thus acts like a stabilizing force, being able to sample and represent rare events for more generations.

Model 2, the Layered Discrete Distribution Model, mainly captures late-stage collapse where the structure of the distribution itself becomes distorted. By linking a task-frequency distribution π with task-specific complexity distributions P , the model reveals how recursive training can progressively reshape the overall task landscape. Model 2 shows how regions of the space may flatten, concentrate, or fragment, depending on which tasks receive synthetic updates. The simulations we ran demonstrate how in a situation where half of the input data is AI-generated content, the task-space collapses to a set of delta Dirac-like functions localized around the Tasks each AI model was trained on. We found that this collapse happens relatively quickly, with significant trends realized in less than 30 generations in most cases. These outcomes align closely with the “late collapse” behaviors reported by Shumailov et al., showing how data distributions collapse to singular sets of points defined by the specifications of the corresponding AI model.

Overall, the 2 models show that collapse is not a single phenomenon but a multi-stage process. Early collapse arises from statistical distortion, tail shrinkage, and information-theoretic drift, well captured by the Markov-chain operator in Model 1. Late collapse arises from structural reshaping of the space itself, well captured by the task-complexity feedback loop in Model 2. These combined findings help us see the full picture of how recursive AI training behaves. More importantly however, it shows what parameters are key for stability of the system, those being synthetic-data fraction, diversity of models within the ecosystem, and the breadth of domain coverage. These allow insight on how future AI training pipelines could be designed to mitigate collapse. While simplified, these models emphasize a key conclusion: the long-term stability of AI ecosystems depends on sustained access to human-generated data and structural diversity across models and tasks.

4.1 Limitations and Future Work

Given the number of assumptions and simplifications made in the creation of these models, there are numerous limitations to take into account. Regarding Model 2, the decision of which tasks, complexities, and distributions to implement was somewhat arbitrary; as there is no practical output from each AI model to show how they are affected by changes to the sample data, it is difficult to directly see the effects of the altered distributions. Additionally, the assumption that all AI models are trained the same way is not representative of reality as there are many different types of models that may be impacted differently by changes to sampling data in terms of their output.

Building on these limitations, future extensions of this work could include a focus on functional expressivity error and functional approximation error, as both our analysis as well as the analysis conducted by Shumailov et al. did not focus much on these. Additionally, connecting practical output such as text-generation or image-generation to our models could help better visualize the effects of model collapse. Overall, although our models seem to be helpful in modeling both early and late-stage collapse, including more variables and specifications could help better mimic model collapse in reality.

5. References

Facing Reality? Law Enforcement and the Challenge of Deepfakes. (2024).
https://www.europol.europa.eu/cms/sites/default/files/documents/Europol_Innovation_Lab_Facing_Reality_Law_Enforcement_And_The_Challenge_Of_Deepfakes.pdf

Shumailov, I., Shumaylov, Z., Zhao, Y., Papernot, N., Anderson, R., and Gal, Y. (2024). AI models collapse when trained on recursively generated data. *Nature*, 631(8022), 755–759.
<https://doi.org/10.1038/s41586-024-07566-y>

Thompson, B., Dhaliwal, M.P., Frisch P., Domhan, T., Federico, M. (2025). A Shocking Amount of the Web is Machine Translated: Insights from Multi-Way Parallelism. *arXiv preprint arXiv:2401.05749*.

6. Appendices

Model 1: Modified Markov-Chain Ecosystem Model

```
%% ai_ecosystem_plots.m
% Multi-model recursive training with human + AI mix
% Equations:
%  $Q_{AI,n} = \text{mean}_i Q_{\{i,n\}}$ 
%  $P_n = (1-\alpha) P_0 + \alpha Q_{AI,n}$ 
%  $Q_{\{i,n+1\}} = F_i(P_n)$  (sampling + smoothing + fidelity blend)
%
% This script:
% 1) Runs a grid of simulations and saves CSVs.
% 2) Produces three comparison plots required for the report:
%   A) Single-model vs Multi-model (vary M at fixed  $\alpha$ )
%   B) Different  $\alpha$  (human fraction) (vary  $\alpha$  at fixed M)
%   C) Different model diversity (correlated vs diverse errors at fixed M,  $\alpha$ )
%
% Outputs:
% grid_timeseries.csv, grid_summary.csv
% figA_kl_Mvary.png, figB_kl_alphavary.png, figC_kl_diversity.png
% (Entropy and Tail plots also saved with matching names)

clc; clear; close all;

%% ----- Parameters -----
V      = 500;          % categories
gens    = 40;          % generations
trials  = 6;           % Monte Carlo trials per setting

alphas  = [1.00 0.95 0.90 0.80]; % for grid runs
Ms      = [1 3 5];     % for grid runs

% Plot baselines
baselineAlpha = 0.90; % used in Plot A (vary M)
baselineM     = 3;    % used in Plot B (vary alpha)
divAlpha     = 0.90; % used in Plot C (diversity)
divM         = 3;    % used in Plot C (diversity)

aiSamplesPerModel = 4000; % AI samples per model/gen
fidelityLow       = 0.92; % model fidelity range
```

```

fidelityHigh = 0.98;
alphaSmooth = 0.5;      % additive smoothing for counts->probs
tauTail = 5e-4;        % tail threshold under P0
rng(42);

%% ----- Human baseline P0 -----
r = (1:V)'; P0 = 1 ./ (r.^1.07); P0 = P0 / sum(P0);
tailMask = P0 <= tauTail;

%% ----- Run grid (M, alpha) -----
TS = []; % timeseries rows: [M, alpha, corr, trial, gen, KL, Entropy, Tail]
for M = Ms
    for alpha = alphas
        TS = [TS; simulate_scenario(P0, tailMask, V, M, alpha, gens, trials, ...
            aiSamplesPerModel, fidelityLow, fidelityHigh, alphaSmooth, 'corr', 0.0)];
    %#ok<AGROW>
    end
end

% Diversity experiment (C): same M,  $\alpha$  but different correlation in model errors
TS = [TS; simulate_scenario(P0, tailMask, V, divM, divAlpha, gens, trials, ...
    aiSamplesPerModel, fidelityLow, fidelityHigh, alphaSmooth, 'corr', 1.0)];
%#ok<AGROW>
TS = [TS; simulate_scenario(P0, tailMask, V, divM, divAlpha, gens, trials, ...
    aiSamplesPerModel, fidelityLow, fidelityHigh, alphaSmooth, 'corr', 0.0)];
%#ok<AGROW>

% Save timeseries
T = array2table(TS, 'VariableNames', ...
    {'M', 'alpha', 'corr', 'trial', 'gen', 'KL', 'Entropy', 'Tail'});
writetable(T, 'grid_timeseries.csv');

%% ----- Summaries -----
S = summarize_grid(T);
writetable(S, 'grid_summary.csv');

%% ----- Plot A: Single vs Multi (vary M at fixed  $\alpha$ ) -----
subA = T(T.alpha==baselineAlpha & T.corr==0, :);
plot_metric_by('KL', subA, 'M', sprintf('A: Quality Drift (KL) --  $\alpha$ =%.2f', baselineAlpha),
    'figA_kl_Mvary.png');

```

```

plot_metric_by('Entropy', subA, 'M', sprintf('A: Entropy --  $\alpha$ =%.2f', baselineAlpha),
'figA_entropy_Mvary.png');
plot_metric_by('Tail', subA, 'M', sprintf('A: Tail Coverage --  $\alpha$ =%.2f', baselineAlpha),
'figA_tail_Mvary.png');

%% ----- Plot B: Different  $\alpha$  (vary  $\alpha$  at fixed M) -----
subB = T(T.M==baselineM & T.corr==0, :);
plot_metric_by('KL', subB, 'alpha', sprintf('B: Quality Drift (KL) -- M=%d', baselineM),
'figB_kl_alphavary.png');
plot_metric_by('Entropy', subB, 'alpha', sprintf('B: Entropy -- M=%d', baselineM),
'figB_entropy_alphavary.png');
plot_metric_by('Tail', subB, 'alpha', sprintf('B: Tail Coverage -- M=%d', baselineM),
'figB_tail_alphavary.png');

%% ----- Plot C: Diversity (corr=1 vs 0) -----
subC = T(T.M==divM & T.alpha==divAlpha, :);
% Map corr values to names
subC.group = strings(height(subC),1);
subC.group(subC.corr==1) = "Identical models (highly correlated)";
subC.group(subC.corr==0) = "Diverse models (low correlation)";
plot_metric_by_groupname('KL', subC, 'group', ...
    sprintf('C: Quality Drift (KL) -- diversity, M=%d,  $\alpha$ =%.2f', divM, divAlpha),
'figC_kl_diversity.png');
plot_metric_by_groupname('Entropy', subC, 'group', ...
    sprintf('C: Entropy -- diversity, M=%d,  $\alpha$ =%.2f', divM, divAlpha),
'figC_entropy_diversity.png');
plot_metric_by_groupname('Tail', subC, 'group', ...
    sprintf('C: Tail Coverage -- diversity, M=%d,  $\alpha$ =%.2f', divM, divAlpha),
'figC_tail_diversity.png');

disp('Saved: grid_timeseries.csv, grid_summary.csv and all PNG figures.');
```

%% ===== Functions =====

```

function TS = simulate_scenario(P0, tailMask, V, M, alpha, gens, trials, ...
    aiSamplesPerModel, fidelityLow, fidelityHigh, alphaSmooth, varargin)
% varargin: 'corr' => correlation in model errors [0..1], default 0
p = inputParser;
addParameter(p,'corr',0.0);
parse(p,varargin{:});
```

```

corrLevel = p.Results.corr;

aiTotal = M * aiSamplesPerModel;
H = 0; if alpha < 1, H = round(aiTotal*(1-alpha)/alpha); end

TS = [];
for t = 1:trials
    % --- Initialize models near P0 with controllable correlation ---
    % Create a common noise component to induce correlation
    common_noise = dirichlet_rnd(ones(V,1));
    Q = cell(M,1);
    for i = 1:M
        indep_noise = dirichlet_rnd(ones(V,1));
        noise = corrLevel*common_noise + (1-corrLevel)*indep_noise;
        noise = noise / sum(noise);
        Qi = 0.97*P0 + 0.03*noise;
        Qi = Qi / sum(Qi);
        Q{i} = Qi;
    end
    % Correlate fidelities if corrLevel>0
    base_beta = (fidelityLow + fidelityHigh)/2;
    betas = base_beta*ones(M,1) + (1-corrLevel)*(fidelityHigh-base_beta)*(-1 + 2*rand(M,1));
    betas = min(max(betas, fidelityLow), fidelityHigh);
    priors = repmat({ones(V,1)/V}, M, 1);

    for g = 0:gens-1
        % ---- Equations ----
        Qmat = cat(2, Q{:}); % V x M
        QAI = mean(Qmat, 2); % V x 1
        Pn = (1 - alpha)*P0 + alpha*QAI;
        Pn = Pn / sum(Pn);

        % ---- Metrics ----
        KL = kl_div(P0, Pn);
        ENT = entropy_vec(Pn);
        TAIL = sum(Pn(tailMask)) / sum(P0(tailMask));
        TS = [TS; M, alpha, corrLevel, t, g, KL, ENT, TAIL]; %#ok<AGROW>

        % ---- Empirical pool ----
        if alpha < 1

```

```

        counts_h = mnrnd(H, P0');
    else
        counts_h = zeros(V,1);
    end
    counts_ai = zeros(V,1);
    for i = 1:M
        counts_ai = counts_ai + mnrnd(aiSamplesPerModel, Q{i}');
    end
    pool = counts_h + counts_ai;
    emp = (pool + alphaSmooth) ./ (sum(pool) + alphaSmooth*V);

    % ---- Retrain models ----
    for i = 1:M
        Qi_next = betas(i)*emp + (1 - betas(i))*priors{i};
        Q{i} = Qi_next / sum(Qi_next);
    end
end
end
end

function S = summarize_grid(T)
% Average across trials and classify outcome for each (M, alpha, corr)
keys = unique(T(:, {'M', 'alpha', 'corr'}));
rows = [];
for r = 1:height(keys)
    M = keys.M(r); a = keys.alpha(r); c = keys.corr(r);
    sub = T(T.M==M & T.alpha==a & T.corr==c, :);
    agg = groupsummary(sub, 'gen', 'mean', {'KL', 'Entropy', 'Tail'});
    slope = last_half_slope(agg.gen, agg.mean_KL);
    ent_drop = agg.mean_Entropy(ceil(height(agg)/2)) - agg.mean_Entropy(end);
    tailf = agg.mean_Tail(end);
    if slope > 0.01 || tailf < 0.6 || ent_drop > 0.05
        lbl = "collapse";
    elseif (slope > 0.002 && slope <= 0.01) || (tailf >= 0.6 && tailf < 0.9)
        lbl = "slow_decline";
    else
        lbl = "stable";
    end
    rows = [rows; M, a, c, slope, tailf, string(lbl)]; %#ok<AGROW>
end

```

```
S = table(rows(:,1), rows(:,2), rows(:,3), rows(:,4), rows(:,5), rows(:,6), ...
    'VariableNames', {'M','alpha','corr','KL_slope','Tail_final','label'});
end
```

```
function plot_metric_by(metric, Tsub, varyField, ttl, outpng)
% Plot mean metric vs generation for groups defined by varyField
% metric in {'KL','Entropy','Tail'}
cats = unique(Tsub.(varyField));
figure; hold on;
for k = 1:numel(cats)
    gsub = Tsub(Tsub.(varyField)==cats(k), :);
    agg = groupsummary(gsub, 'gen', 'mean', metric);
    plot(agg.gen, agg.(['mean_' metric]), 'LineWidth', 1.6, ...
        'DisplayName', sprintf('%s=%s', varyField, num2str(cats(k))));
end
xlabel('Generation');
ylabel(metric_label(metric));
title(ttl);
legend('show'); grid on;
saveas(gcf, outpng);
end
```

```
function plot_metric_by_groupname(metric, Tsub, groupField, ttl, outpng)
% Plot using a string group field already present (e.g., "group")
groups = unique(Tsub.(groupField));
figure; hold on;
for k = 1:numel(groups)
    gsub = Tsub(Tsub.(groupField)==groups(k), :);
    agg = groupsummary(gsub, 'gen', 'mean', metric);
    plot(agg.gen, agg.(['mean_' metric]), 'LineWidth', 1.6, ...
        'DisplayName', groups(k));
end
xlabel('Generation'); ylabel(metric_label(metric));
title(ttl); legend('show'); grid on; saveas(gcf, outpng);
end
```

```
function txt = metric_label(metric)
switch metric
    case 'KL',    txt = 'KL(P0 || Pn)';
    case 'Entropy', txt = 'Entropy(Pn)';
```

```

        case 'Tail',    txt = 'Tail coverage (Pn tail / P0 tail)';
        otherwise,     txt = metric;
    end
end

%% ----- Math helpers -----
function y = kl_div(p,q)
    eps = 1e-12; p = max(p,eps); q = max(q,eps);
    y = sum(p .* (log(p) - log(q)));
end

function h = entropy_vec(p)
    p = p(p>0); h = -sum(p .* log(p));
end

function x = dirichlet_rnd(a)
    g = gamrnd(a,1); x = g ./ sum(g);
end

function slope = last_half_slope(g, y)
    g = g(:); y = y(:); k = ceil(numel(g)/2);
    A = [g(k:end) ones(numel(g)-k+1,1)];
    coeff = A \ y(k:end);
    slope = coeff(1);
end

```

Model 2: Layered Discrete Distribution Model

```

clc
clear variables
close all

% ===== Parameters =====

% Tasks
Size = 100;

% Tasks to train on
Tasks = [20:80];

```

```

% Number of Generations
Stop = 100;

% Size of Training Data Sets
M = 1000;

% ===== Initial Conditions =====

% Initial Task Distribution (Prior)
SpacePdf = cell(1,Stop+1);
SpacePdf{1} = binopdf(1:Size,Size,0.5);

% Initial Difficulty/Task Distribution (Likelihood)
TaskSetUp = cell(1,Stop+1);
TaskPdf = cell(1,Size);

for i = 1:Size
    TaskPdf{i} = TaskSetUp;
    TaskPdf{i}{1} = geopdf(1:5,1-SpacePdf{1}(i));
end

% Transfer Functions
    % Testing with only sampling errors, same function for all AIs
F = @(x) x;

% ===== Solver =====

for j = 1:Stop

    % Generation j

    for T = Tasks
        % Distribution for drawing
        %Dist = (2.*TaskPdf{T}{j}.*SpacePdf{j}(T) + TaskPdf{T-1}{j}.*SpacePdf{j}(T-1) +
TaskPdf{T+1}{j}.*SpacePdf{j}(T+1))./sum((2.*TaskPdf{T}{j}.*SpacePdf{j}(T) +
TaskPdf{T-1}{j}.*SpacePdf{j}(T-1) + TaskPdf{T+1}{j}.*SpacePdf{j}(T+1)));
        Dist = TaskPdf{T}{j};

        % Creating Samples
        Dataset = randsample(1:5,M,true,Dist);

        % Estimating Distribution

```

```

counts = histcounts(Dataset,5);
eDist = counts/M;
p = mle(Dataset,'distribution','Geometric');

% Transfer Function
p = F(p);
%eDist = geopdf(1:5,p);

% Generating Content by AI and by Humans
TaskPdf{T}{j+1} = 0.5.*eDist + 0.5.*TaskPdf{T}{j};
%TaskPdf{T-1}{j+1} = 0.5.*eDist + 0.5.*TaskPdf{T}{j};
%TaskPdf{T+1}{j+1} = 0.5.*eDist + 0.5.*TaskPdf{T}{j};

end

% Regenerating Task Space
for T = 1:Size
    if isempty(TaskPdf{T}{j+1})
        TaskPdf{T}{j+1} = TaskPdf{T}{j};
    end
    SpacePdf{j+1}(T) = TaskPdf{T}{j}(1);
end
SpacePdf{j+1} = SpacePdf{j+1}/sum(SpacePdf{j+1});

end

% ===== Plotting =====

figure
tiledlayout(1,2)
for i = [1,Stop+1]
    nexttile
    plot(SpacePdf{i})
    xlabel('Function')
    ylabel('Probability')
    title(['Task Space, Generation: ', int2str(i-1)])
end

for j = 20
    figure
    tiledlayout(2,1)
    for i = [1,Stop+1]
        nexttile

```

```
    bar(TaskPdf{j} {i})
    axis([1,5,0,1])
    xlabel('Diversity')
    ylabel('Probability')
    title(['Task Complexity, Generation: ', int2str(i-1)])
end

end
```